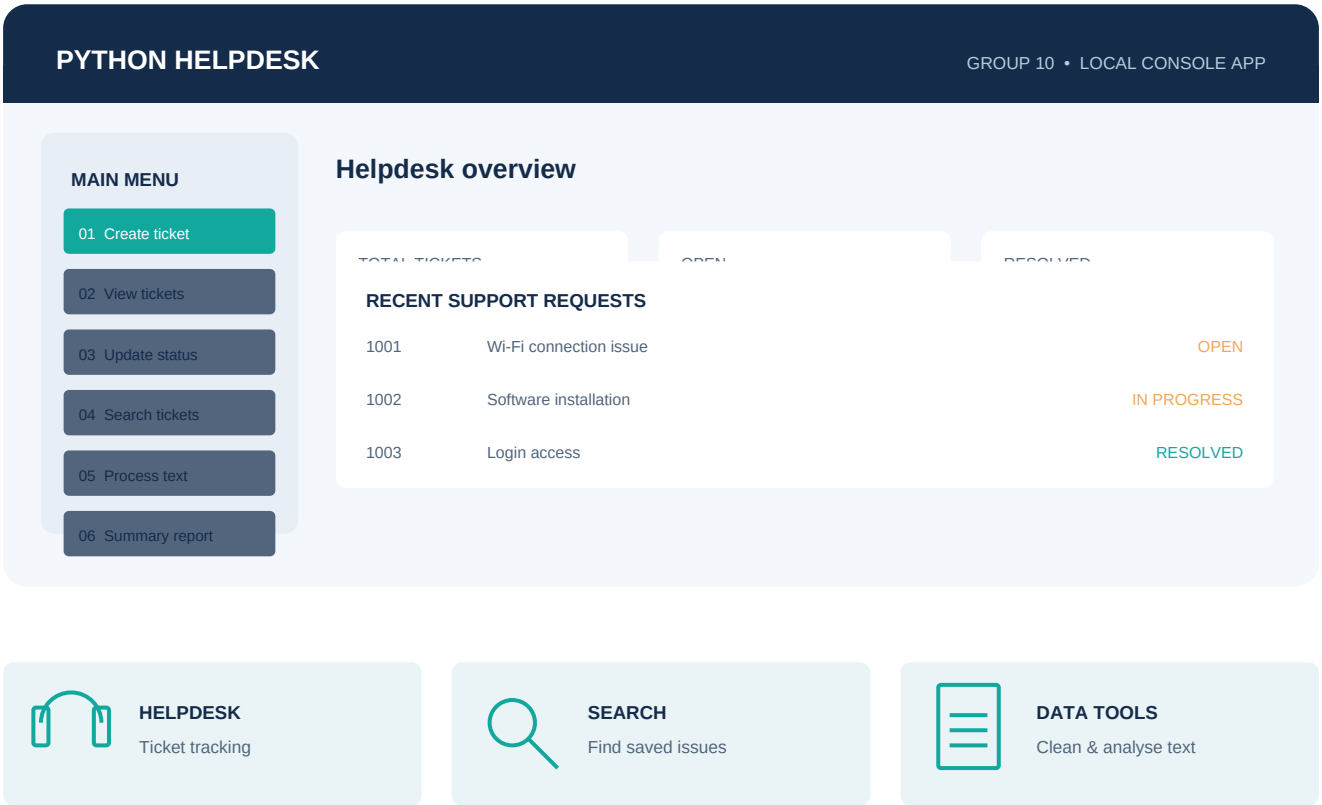
BUDGE BUDGE INSTITUTE OF TECHNOLOGY, KOLKATA

Department of Computer Science & Engineering

IT WORKSHOP (PYTHON PROGRAMMING)
MINI PROJECT REPORT

INTEGRATED PYTHON HELPDESK & DATA PROCESSING SUITE

A menu-driven application for support-ticket management and text-data processing



HELPDESK
Ticket tracking

SEARCH
Find saved issues

DATA TOOLS
Clean & analyse text

PROJECT DETAIL	ENTRY
Academic Year	2026–2027
Group Number	10
Subject Code	PCC-CS393
Department	Computer Science & Engineering
Group Members	Kakoli Mandal Swastika Brahma Rupanjana Ghosh Ankita Manna
Submission Date	30 October 2026

01 | Project Overview

Project Title

Integrated Python Helpdesk & Data Processing Suite — a Python-based, menu-driven application for recording, searching and updating helpdesk tickets while also cleaning and analysing text data.

Abstract

The application provides a simple way to record support requests, assign each request a unique ticket ID, view saved tickets, update ticket status and search ticket records. Its text-processing feature normalises extra whitespace and calculates word and character counts. Ticket records and processed-text summaries are stored in CSV files, allowing information to remain available after the program closes. The project demonstrates functions, loops, conditional statements, lists, dictionaries, CSV file handling, input validation, date-time handling and exception handling using Python's standard library.

Introduction

Educational institutions and small organisations receive support requests about login access, network connectivity, software and other technical issues. When these requests are written in notebooks or informal messages, it can become difficult to identify their priority, check their current status or find an earlier report. Separately, text entered into forms or reports may contain inconsistent spacing and may need basic analysis.

This project combines two related utilities in one application: a lightweight helpdesk ticket manager and a text-data processing tool. Its console menu gives users a consistent set of operations, while CSV files provide straightforward local storage. The project is an academic demonstration, not a production multi-user support platform.

Problem Statement

Manual tracking of support requests can lead to missing details, repeated entries and limited visibility into whether an issue is Open, In Progress or Resolved. Searching records manually also takes time. In addition, raw text may contain extra whitespace and require basic word and character analysis. The project addresses these needs through structured ticket records, keyword search, status updates and a simple text-cleaning workflow.

02 | Objectives & System Analysis

Objectives

- Create a unique numeric ID for every new helpdesk ticket.
- Store the user's name, issue category, priority, description, status and creation timestamp.
- Display all saved tickets in a readable format.
- Update a ticket to Open, In Progress or Resolved.
- Search ticket records using an ID, name, category or keyword.
- Normalise whitespace in entered text and calculate character and word counts.
- Save processed-text summaries to a CSV file.
- Generate summary counts grouped by ticket status and priority.
- Demonstrate core Python programming concepts with input validation and exception handling.

Existing System

EXISTING APPROACH	TYPICAL LIMITATION
Paper registers / notebooks	Records may be difficult to search, update and summarise.
Unstructured messages or group chats	Issue details and current resolution status can become scattered.
Manual ticket tracking in spreadsheets	Requires repeated manual entry and consistent formatting.
Manual text counting	Time-consuming and prone to inconsistent whitespace treatment.

Proposed System

The application presents a main menu with six operations and an Exit option. Ticket operations load records from **helpdesk_tickets.csv**, perform the selected task and save changes when required. Text processing cleans the entered text, calculates counts and appends a summary to **processed_data.csv**. The summary-report option calculates totals by status and priority from saved ticket records.

Scope of the Project

Included: ticket creation, viewing, status update, keyword search, text cleaning, word/character counting, CSV storage, summary counts, input validation and basic file-error handling.

Not included: graphical user interface, user authentication, role-based access, remote database, multi-user access, email notifications, automatic ticket assignment or network-based synchronisation. These are possible future enhancements.

03 | Requirements & Data Storage

Hardware and Software Requirements

AREA	REQUIREMENT
Programming language	Python 3.10 or later (as specified in the project report).
Libraries	Python standard library: csv, os and datetime.
Operating system	Windows, macOS or Linux with a supported Python installation.
Hardware	A basic computer capable of running Python and a terminal/IDE.
IDE / editor	Any Python-compatible editor or IDE, such as IDLE, VS Code or PyCharm.
Input / Output	Keyboard input through the console; text displayed in the terminal.
Runtime data files	helpdesk_tickets.csv and processed_data.csv.
Execution command	python 10_CSE.py

Files Used by the Application

FILE	PURPOSE	WRITE BEHAVIOUR
helpdesk_tickets.csv	Stores ticket ID, name, category, priority, description, status and created_at.	Rewritten when tickets are created or updated.
processed_data.csv	Stores original text, cleaned text, character count, word count and processed_at.	New processing results are appended; header is written when needed.

Installation and Execution

- Install Python 3.10 or later.
- Save the supplied source file **10_CSE.py** in a folder.
- Open a terminal or command prompt in that folder.
- Run **python 10_CSE.py**.
- Select an operation from the displayed menu and follow the prompts.
- The CSV files are created in the program’s working folder when data is saved.

No third-party package installation is required because the program uses Python’s standard library.

04 | System Design

The diagrams in this section represent the logical structure and workflows of the supplied console application. They describe program behaviour visible in the Python source code.

User Interface Concept

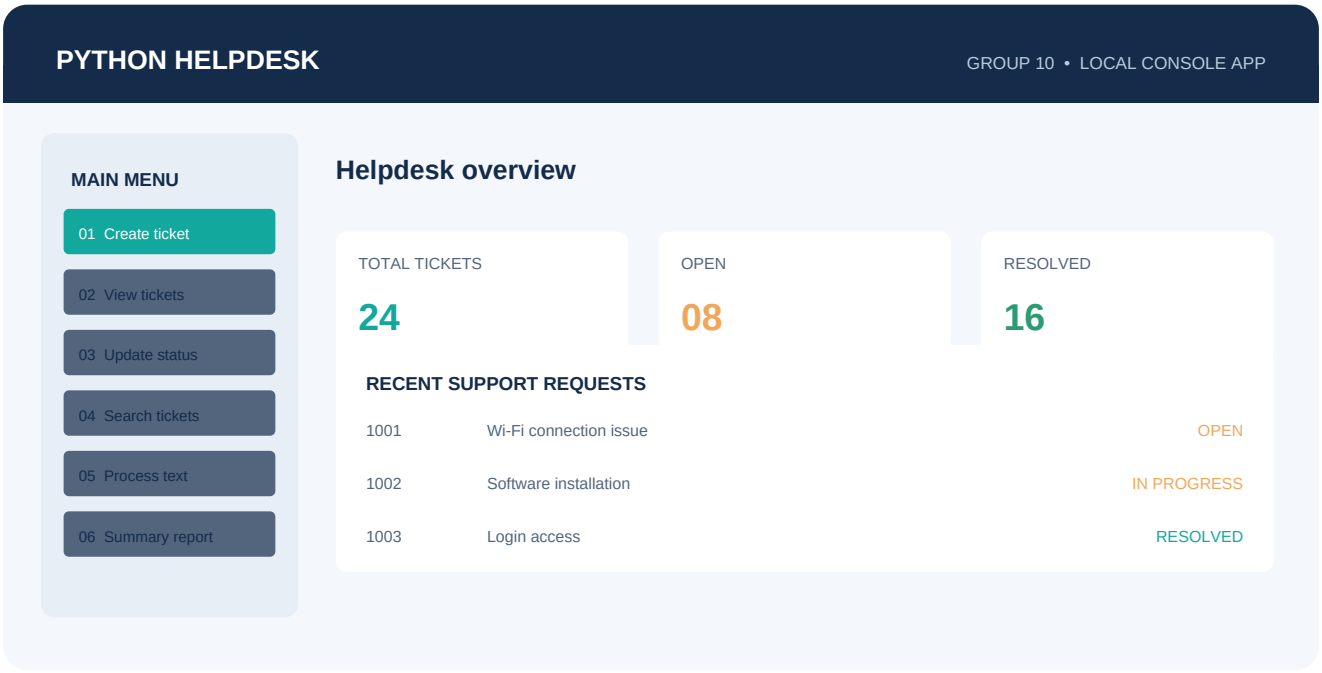
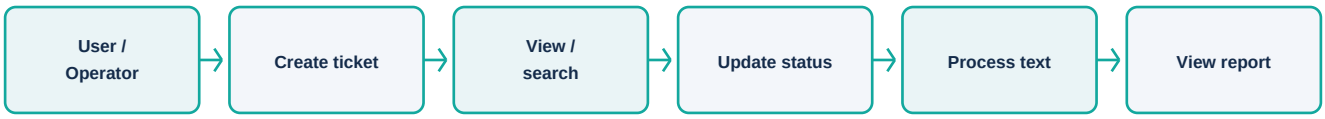


Figure 1. Conceptual visual layout of the helpdesk overview. This illustration is a design mock-up, not a screenshot of an implemented GUI.

Use Case Diagram



The primary actor is the User / Helpdesk Operator. The user interacts with the console menu to create, view, update and search tickets, process text data, and display a summary. CSV files act as local storage.

Conceptual Data Structure

RECORD	FIELDS / ATTRIBUTES
Ticket	ticket_id, name, category, priority, description, status, created_at
Processed Text Summary	original_text, cleaned_text, character_count, word_count, processed_at

05 | Architecture & Program Flow

In-memory Representation

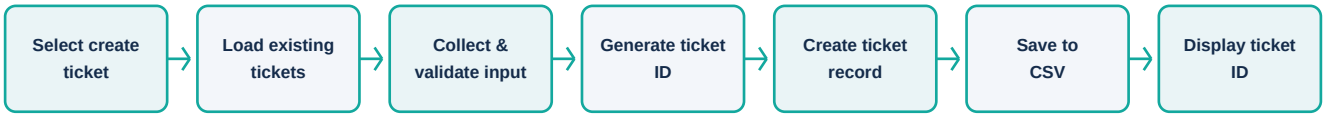
The program uses a list of dictionaries to represent ticket records in memory. Each dictionary stores the fields for one ticket. The CSV module converts these records to and from rows in the persistent ticket file. This is a simple file-based data model; it is not a relational database.

Conceptual Architecture



Figure 2. High-level program components and local data storage.

Activity Diagram — Create Ticket



The operation loads existing records, validates the entered details, generates the next numeric ID, creates a ticket with default status Open and the current timestamp, saves the records and displays the new ID.

Main Menu Logic

The menu repeats until the user chooses Exit. Options 1–6 call the corresponding ticket, text-processing or reporting functions; option 0 ends the menu loop. Invalid menu input is handled through integer and range validation.

06 | Algorithms & Pseudocode

Algorithm A — Create a Helpdesk Ticket

- Start the create-ticket operation.
- Load existing ticket records from the CSV file.
- Find the highest numeric ticket ID and add one to generate the next ID.
- Ask the user for a name, issue category, priority and description.
- Reject blank text fields and accept only a priority from 1 to 3.
- Convert the selected priority number to Low, Medium or High.
- Create a ticket record with status Open and the current date-time.
- Append the record to the in-memory ticket list and save all records to CSV.
- Display the new ticket ID if saving succeeds; otherwise display an error.

Algorithm B — Process Text Data

- Ask the user to enter non-empty text.
- Clean the text by splitting on whitespace and joining the parts with one space.
- Split the cleaned text into words and calculate the number of words.
- Calculate the character count using the length of the cleaned text, including spaces.
- Create a summary containing original text, cleaned text, both counts and a timestamp.
- Append the summary to processed_data.csv, writing a header if the file is new or empty.
- Display the cleaned text, word count, character count and save confirmation.

Pseudocode — Main Menu

```
BEGIN
  REPEAT
    DISPLAY main menu
    INPUT choice and validate integer range 0 to 6
    IF choice = 1: create ticket
    ELSE IF choice = 2: view all tickets
    ELSE IF choice = 3: update ticket status
    ELSE IF choice = 4: search tickets
    ELSE IF choice = 5: process and save text data
    ELSE IF choice = 6: show summary report
    ELSE IF choice = 0: display exit message
  UNTIL choice = 0
END
```

07 | Implementation & Modules

MODULE / FUNCTION	RESPONSIBILITY
read_int()	Reads an integer and repeats the prompt when input is invalid or outside the specified range.
read_nonempty()	Reads text and rejects blank input after trimming whitespace.
load_tickets()	Reads ticket rows from the CSV file; returns an empty list when the file is absent and handles read errors.
save_tickets()	Writes ticket records and the required column headings to helpdesk_tickets.csv.
create_ticket()	Generates the next ID, collects details, assigns default Open status and saves the ticket.
view_tickets()	Displays all saved tickets, including ID, status, name, category, priority, description and creation time.
update_ticket_status()	Finds a ticket by ID, accepts a new status and saves the updated records.
search_tickets()	Performs a case-insensitive keyword search across ticket field values.
process_text_data()	Normalises whitespace, calculates counts and appends the result to processed_data.csv.
show_report()	Displays total ticket count and counts by status and priority.
main()	Displays the menu and calls the selected function until the user chooses Exit.

Important Python Concepts Used

Functions divide the application into reusable operations. Lists and dictionaries store ticket records in memory. Loops repeat the menu and validation prompts. Conditional statements select menu actions and map numeric choices to labels. CSV handling stores records in a structured text format. Exception handling manages invalid integer input and file errors. datetime records when a ticket or text summary is created.

Text Processing Illustration

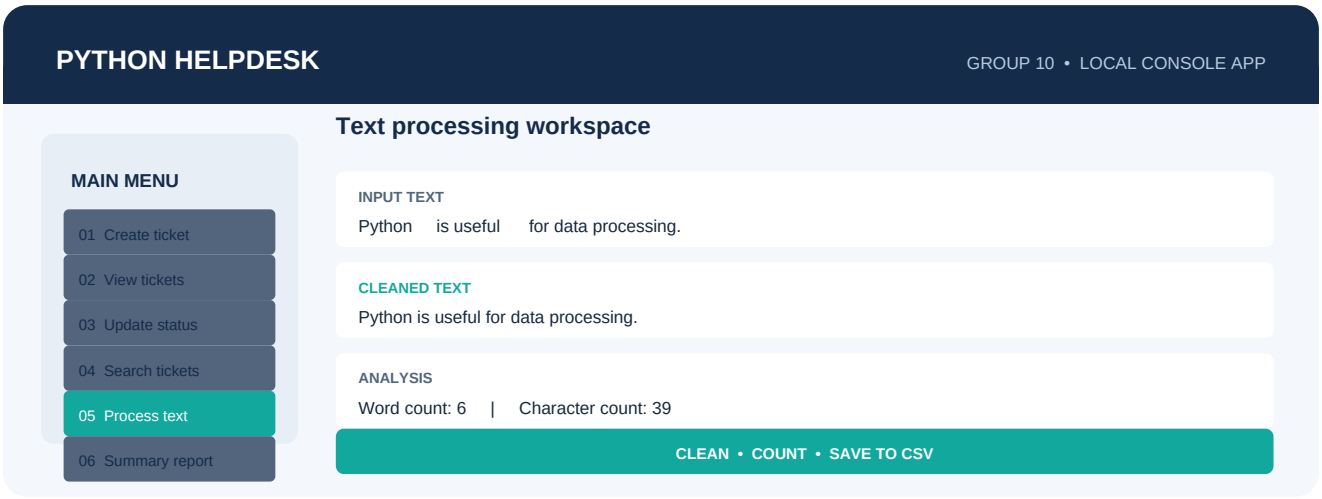


Figure 3. Illustrative text-cleaning workspace showing the type of information produced by the processing function.

08 | Code & Sample Execution

Text Cleaning and Counting

```
cleaned = " ".join(text.split())\nwords = cleaned.split()\ncharacter_count = len(cleaned)\nword_count = l
```

The split-and-join operation collapses runs of whitespace into a single space and removes leading/trailing whitespace. The character count is based on the cleaned text and includes spaces.

Priority Selection

```
priority = {1: "Low", 2: "Medium", 3: "High"}[priority_number]
```

The dictionary maps the validated numeric priority choice to its descriptive label.

Sample Execution — Ticket Creation

```
INTEGRATED PYTHON HELPDESK & DATA PROCESSING SUITE\n1. Create helpdesk ticket\n2. View all tickets\n3. Up
```

Illustrative example: ticket ID 1001 is produced when no earlier numeric ticket IDs exist. The actual next ID depends on the records already saved in the CSV file.

Sample Execution — Text Processing

```
Enter text to clean and analyse:       Python     is useful\nCleaned text: Python is useful\nCharacter c
```

09 | Testing & Results

The following test cases are derived from the implemented menu options and validation logic. Execute them on the target computer and record Pass or Fail from the observed result before submission.

ID	TEST / INPUT ACTION	EXPECTED RESULT	STATUS
TC-01	Choose option 1 and enter valid ticket details.	Ticket saved with a new ID and Open status.	To verify
TC-02	Enter letters instead of a menu number.	Validation message appears and input is requested again.	To verify
TC-03	Leave the student name blank.	Blank input is rejected; prompt repeats.	To verify
TC-04	View tickets when the ticket CSV does not exist.	Displays “No tickets found.”	To verify
TC-05	Search using a matching keyword.	Matching ticket summary is displayed.	To verify
TC-06	Update the status of an existing ticket ID.	Selected status is saved to the CSV file.	To verify
TC-07	Process text containing extra spaces.	Whitespace is normalised; word and character counts are shown.	To verify
TC-08	Enter a ticket ID that does not exist.	Displays “Ticket ID not found.”	To verify
TC-09	Choose an invalid priority outside 1–3.	Range validation message appears; prompt repeats.	To verify
TC-10	Run summary report with saved tickets.	Displays total tickets and counts by status and priority.	To verify

Observed / Expected Behaviour

The source code defines the menu operations, validation rules, CSV read/write routines and report calculations. This report does not claim that the test cases have been executed on the user’s computer. The Status column is therefore intentionally marked “To verify” and should be updated after practical testing.

Limitations

The application is a local, single-user console program. It does not provide a graphical interface, user authentication, access permissions or remote synchronisation. CSV files are appropriate for a small demonstration but are not designed for concurrent multi-user access or large-scale data management. Search is performed over the records loaded into memory.

10 | Results & Future Scope

Representative Application Screens

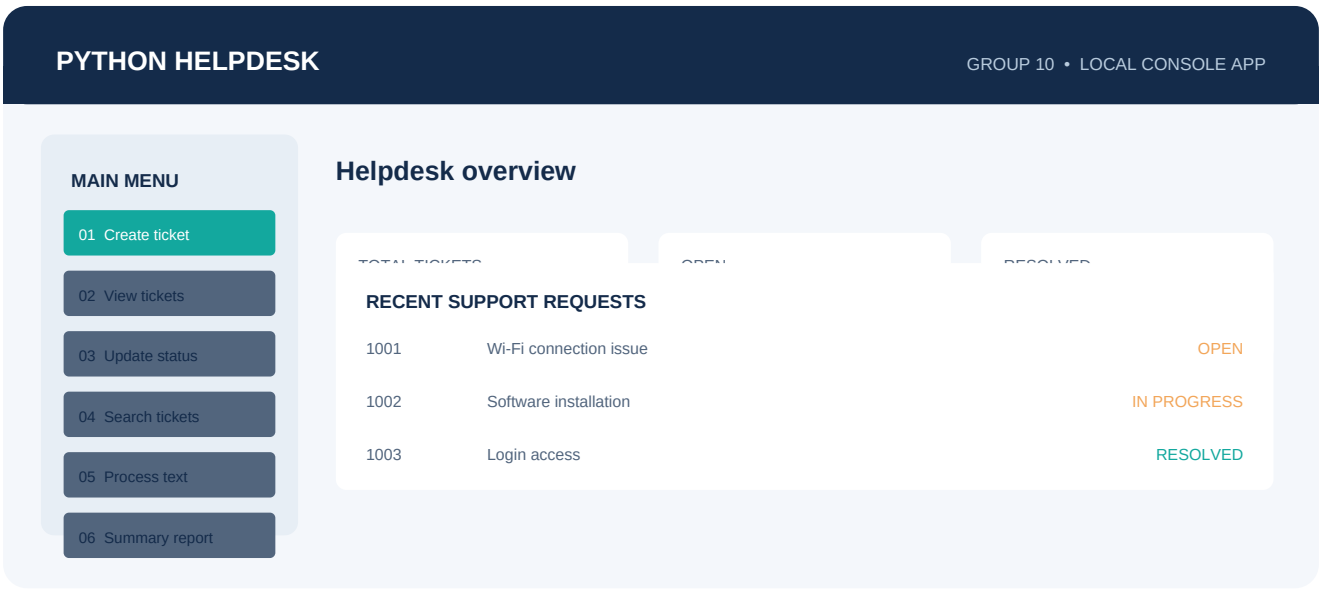


Figure 4. Helpdesk dashboard concept — ticket overview and recent requests.

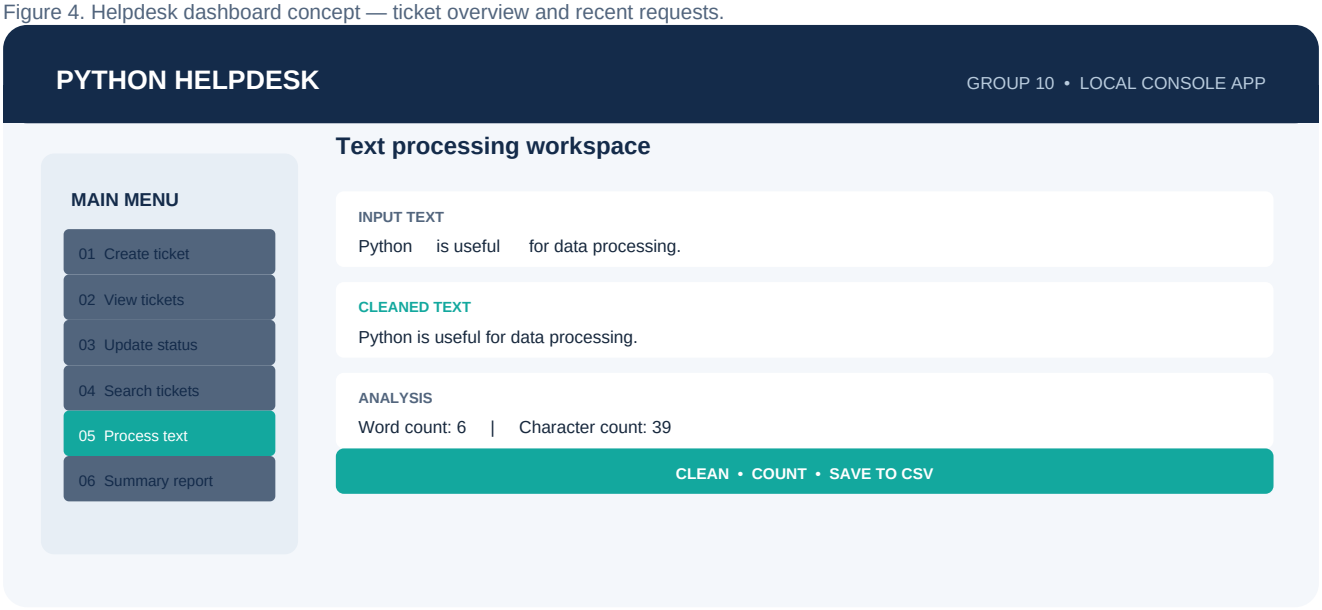


Figure 5. Text processing concept — cleaned text and calculated counts.

These are illustrative vector mock-ups created to improve report presentation. They are not screenshots of a graphical interface implemented by the current console application.

Conclusion

The Integrated Python Helpdesk & Data Processing Suite demonstrates how Python can be used to build a small integrated utility for managing support requests and processing text data. The application supports ticket creation, viewing, status updates and keyword searching, while also providing whitespace normalisation, word and character counting, CSV storage and summary reporting. Its modular functions and use of lists, dictionaries, loops, conditional statements, file handling and exception handling make it suitable for demonstrating core Python programming concepts.

Future Scope

- Develop a graphical user interface using Tkinter for easier interaction.
- Replace or supplement CSV storage with SQLite for structured querying and stronger data management.

- Add user login, authentication and role-based permissions.
- Introduce ticket assignment, comments, resolution notes and ticket history.
- Add date filters, category-wise analytics and exportable summary reports.
- Provide email or in-app notifications when ticket status changes.
- Add automated unit tests for input validation, file operations and ticket calculations.
- Improve accessibility and usability with clearer prompts and a more informative dashboard.

11 | References & Project Files

References

1. Python Software Foundation, Python 3 Documentation, https://docs.python.org/3/ (accessed 30 October 2026).

2. Python Software Foundation, csv — CSV File Reading and Writing, https://docs.python.org/3/library/csv.html (accessed 30 October 2026).

3. Python Software Foundation, datetime — Basic Date and Time Types, https://docs.python.org/3/library/datetime.html (accessed 30 October 2026).

4. Python Software Foundation, os — Miscellaneous Operating System Interfaces, https://docs.python.org/3/library/os.html (accessed 30 October 2026).

5. Group 10, 10_CSE.py, supplied project source code and project description (2026).

Appendix — Project Files

FILE	DESCRIPTION
10_CSE.py	Main executable Python source file.
helpdesk_tickets.csv	Created or updated by the program to store helpdesk ticket records.
processed_data.csv	Created or updated by the program to store text-processing summaries.
Mini Project Report	Documentation of objectives, system design, algorithms, modules, testing and future scope.

THANK YOU

GROUP 10 • COMPUTER SCIENCE & ENGINEERING

Kakoli Mandal • Swastika Brahma • Rupanjana Ghosh • Ankita Manna